

Betriebssysteme

- Übung 2: Das Santa-Claus-Problem -

Franz Pawlus
1422169

25. Januar 2026

Aufgabe 1: Semaphore-basierte Implementierung

Für die Umsetzung der ersten Teilaufgabe habe ich ein Python-Skript erstellt und dabei die `threading`-Bibliothek genutzt, da diese native Semaphore bereitstellt.

Da der Fokus des Übungsblatts auf Aufgabe 2 liegen sollte, habe ich mich nach kurzem Ausprobieren an der Logik von Downeys *"Little Book of Semaphores"* orientiert. Zusätzlich habe ich jedoch ein `elfSem`-Semaphore eingeführt, da sich die Lösung so für mich intuitiver umsetzen ließ.

Listing 1: Initialisierung der Semaphore

```
1 # Schützt reindeer_count und elf_count vor Race Conditions
2 mutex = threading.Semaphore(1)
3
4 # Initial auf 0, damit Santa bei .acquire() blockiert (schlft)
5 santaSem = threading.Semaphore(0)
6
7 # Rentiere warten hier auf Santa
8 reindeerSem = threading.Semaphore(0)
9
10 # Elfen warten hier auf Hilfe von Santa
11 elfSem = threading.Semaphore(0)
12
13 # Stellt sicher, dass keine neuen Elfen Gruppen bilden,
14 # solange Santa hilft oder Geschenke verteilt
15 elfTex = threading.Semaphore(1)
```

Zusätzlich zu den Semaphoren habe ich entsprechend zwei Zählervariablen eingeführt (nach Vorbild von Downeys „Hint“), deren Zugriff durch den `mutex` gesteuert wird:

```
1 reindeer_count = 0
2 elf_count = 0
```

Hierbei fungieren `reindeerSem` und `elfSem` als Sammelbecken für die insgesamt r Rentiere und die Gruppe von p Elfen, während diese auf Santa warten.

Besonders kritisch war für mich das Verständnis des `elfTex`-Semaphors. Zunächst vermutete ich, dass ein `mutex` für den Zähler ausreichen würde. Ich erkannte jedoch bei der Implementierung, dass ohne `elfTex` eine *Race Condition* zwischen der Elfengruppe und dem Weihnachtsmann entstehen würde: Wenn während Santas Abwesenheit mehr als p Elfen eintreffen, würde der Zähler den Schwellenwert von p überschreiten. Da Santa im Code explizit auf den Wert `elf_count == p` prüft, würde er eine größere Gruppe ignorieren, was zu einem *Deadlock* führen würde.

`elfTex` dient somit als Barriere, die die Anzahl der Elfen, die sich gleichzeitig im kritischen Bereich befinden dürfen, strikt auf p begrenzt. Alternativ hätte man das Problem auch durch eine `elf_count >= p`-Bedingung lösen können; dies hätte jedoch eine komplexere Logik erfordert, um mit Gruppen umzugehen, die größer als p sind. Den Lösungsansatz von Downey fand ich daher sehr intuitiv (bis auf das Fehlen des `elfSem`, welches ich ergänzt habe).

Aufgabe 2: Implementierung mittels Docker-Container und ZeroMQ

Der wesentliche Unterschied zwischen der Anforderung von Aufgabe 2 und der vorangegangenen Implementierung besteht darin, dass das System nicht mehr auf einem gemeinsamen Adressraum basiert. Stattdessen wird das Problem auf $1 + r + e$ isolierte Docker-Container verteilt, deren einzige Form der Interaktion der nachrichtenbasierte Austausch über ZeroMQ-Sockets ist.

Der Koordinationscontainer (Öffice’)

Um die Komplexität der Verteilung etwas intuitiver zu gestalten, habe ich mich dazu entschieden, einen zentralen Koordinationscontainer zu implementieren. Ich habe diesen `office` genannt.

Zuerst hatte ich eine (funktionierende) Lösung ohne diesen `office`-Container entworfen, empfand diese aber als „unsauber“, da der Code von `santa` sehr aufgebläht wirkte und nicht seiner Rolle als schlafende Ressource entsprach.

Durch die Auslagerung der Gruppenbildung und Warteschlangenverwaltung ist der Code für den Weihnachtsmann schlanker und übersichtlicher geworden. Zudem muss er nicht mehr während des Schlafens das gesamte Netzwerk koordinieren, was die Implementierung wesentlich intuitiver macht.

Listing 2: Implementierung des Santa-Services

```
1 def santa():
2     ctx = zmq.Context()
3     # REP-Socket als Dienstleister
4     socket = ctx.socket(zmq.REP)
5     socket.bind("tcp://*:5556")
6
7     print("Santa: Ich schlafe...", flush=True)
8
9     while True:
10        task = socket.recv_json()
11        print(task)
12        role = task.get("role")
13
14        if role == "reindeer":
15            print("Santa: Wuahhh, schon wieder Weihnachten! Schlitten bereit machen!", flush=True)
16            time.sleep(2)
17        else:
18            print("Santa: Ich helfe den Elfen bei den Geschenken.", flush=True)
19            time.sleep(1)
20
21        socket.send_json({"status": "finished"})
```

Synchrone vs. Asynchrone Lösung

Zunächst experimentierte ich mit einem einfachen **Request-Reply**-Muster und aktivem Polling. In diesem Szenario fragten die Clients (Rentiere und Elfen) periodisch im Ein-Sekunden-Takt ihren Status beim Koordinator bzw. Weihnachtsmann ab. Erhielten sie ein „Wait“, setzten sie das Warten fort; erst bei einer Freigabe führten sie ihre Aktion aus.

Listing 3: Früherer Ansatz: Aktives Polling (Pseudo-Code)

```
1 while not ready:
2     socket.send_json({"role": "elf"})
3     response = socket.recv_json()
4     if response["status"] == "done":
5         ready = True
6     else:
7         time.sleep(1) # Aktives Warten
```

Ich war mit diesem (funktionierenden) Ansatz jedoch nicht zufrieden, da er mit einem erhöhten Netzwerkaufkommen und CPU-Overhead einherging. Zudem bildete meine erste Lösung die besondere „Schlaf-Eigenschaft“ der Semaphore aus Aufgabe 1 nicht korrekt ab.

Um die Effizienz zu steigern, habe ich das Polling verworfen und auf ein asynchron arbeitendes **Router-Dealer-Modell** umgestellt, welches deutlich ressourcenschonender ist. Bei diesem Ansatz werden die Clients echt blockiert: Sie senden nur einmalig ihre Anfrage und treten dann in einen blockierenden Empfangszustand (`socket.recv_json()`) ein. Das Office verwaltet hierbei die spezifischen Identitäten der Clients in einfachen Listen.

Listing 4: Auszug aus der actor.py (Client)

```
1 def run_actor():
2     role = os.getenv("ROLE")
3     my_id = py_socket.gethostname()
4
5     ctx = zmq.Context()
6     socket = ctx.socket(zmq.DEALER) # sendet an ROUTER (office)
7     socket.setsockopt(zmq.IDENTITY, my_id.encode())
8     socket.connect("tcp://office:5555")
9
10    while True:
11        # random sleep
12        wait = random.uniform(5, 15) if role == "reindeer" else
            random.uniform(2, 6)
13        time.sleep(wait)
14
15        # send an office
16        socket.send_json({"role": role})
17
18        # warten (echtes schlafen) auf Antwort. Anders als erste
            Version mit REQ-Socket
19        socket.recv_json()
20        print(f"{'reindeer: ' if role == 'reindeer' else 'elf: '}
            {role.capitalize()} {my_id}: Freigabe erhalten!",
            flush=True)
```

```

21
22     # sleep fuer Schlitten ziehen oder Hilfe bekommen
23     time.sleep(2)

```

Listing 5: Auszug aus der office.py (Koordinator)

```

1  def office():
2      ctx = zmq.Context()
3
4      # kommuniziert mit DEALER-Sockets (Elfen, Rentier)
5      # vorher: REQ/REP-Pattern (polling, aktives warten),
6      # jetzt: DEALER/ROUTER pattern entspricht sleep eines
       semaphores besser
7      client_portal = ctx.socket(zmq.ROUTER)
8      client_portal.bind("tcp://*:5555")
9
10     queues = {"reindeer": [], "elf": []} # speichert eingehende
       Anfragen
11     limits = {"reindeer": 9, "elf": 3} # limits der
       warteschlangen
12
13     # REQ-Socket, weil santa eine exklusive Ressource ist
14     santa_link = ctx.socket(zmq.REQ)
15     santa_link.connect("tcp://santa:5556")
16
17     print("Office: □Biep□Biep□Bieeeeeep!□Koordination□gestartet...
       ", flush=True)
18
19     while True:
20         msg_parts = client_portal.recv_multipart()
21         identity = msg_parts[0]
22         payload = msg_parts[1]
23
24         msg = json.loads(payload)
25         role = msg.get("role")
26
27         if role in queues:
28             if identity not in queues[role]:
29                 queues[role].append(identity)
30                 print(f"Office: □{role.capitalize()}□{identity.
                       decode()}□registriert□({len(queues[role])}/{
                       limits[role]})", flush=True)
31
32                 if len(queues[role]) == limits[role]: # Limit
                       erreicht
33                     print(f"Office: □Gruppe□({role})□voll.□Wecke□Santa
                           ...", flush=True)
34
35                     santa_link.send_json({"role": role}) # send und
                       warte auf antwort
36                     santa_link.recv()
37

```

```

38         for client_id in queues[role]: # Rueckmeldung an
           Clients
39             client_portal.send_multipart(
40                 [client_id, json.dumps({"status": "done"
41                                         }).encode()]
42             )
43
44         queues[role] = []
45         print(f"Office: Gruppe({role}) fertig.", flush=
46               True)
47
48         # Gedenkpause fuer sauberes Logging im Terminal
49         time.sleep(0.5)

```

REQ- vs. DEALER-Sockets

Trotz des Wechsels auf asynchrone Dealer-Sockets für die Clients bin ich für die Steuerung des Weihnachtsmanns bewusst bei einem synchronen **Request-Socket** geblieben. Dies hat zur Folge, dass der Koordinator blockiert, solange er auf die Antwort von Santa wartet.

Santa wird somit als exklusive Ressource behandelt, die erst ihre Aufgabe (Helfen oder Fliegen) beenden muss, bevor das Office die nächste Gruppe koordiniert. Man hätte dies zwar ebenfalls mit einem Dealer-Socket asynchron gestalten können, um die Verfügbarkeit des Koordinators zu erhöhen, ich empfand die synchrone Variante jedoch als intuitiver, da sie die Semaphor-Logik aus Aufgabe 1 präziser im Netzwerk imitiert. Zudem bildet sie besser ab, dass Santa nur eine Aufgabe zurzeit wahrnehmen kann.

Schwierigkeiten: Logging-Kausalität

Eine große Schwierigkeit bei der Implementierung lag in der Kausalität der Docker-Logs. Da 21 Container unabhängige Datenströme parallel verarbeiten, erschienen die Anzeigen in den Terminals oft nicht in der logisch richtigen Reihenfolge. Beispielsweise wurde Santas Arbeitsbericht im Log manchmal vor der Registrierung der auslösenden Elfen angezeigt.

Um dies zu beheben und die Korrektheit der Lösung verifizieren zu können, habe ich kleine Verzögerungen (**sleeps**) in den Koordinator eingebaut. Diese geben den Containern ausreichend Zeit, die Ereignisse chronologisch korrekt darzustellen.